Лабораторна робота №3

Тема: Прикладні бібліотеки Python для побудови веб-застосунків

Мета: Навчитися створювати Web API за допомогою FastAPI, визначати маршрути та користуватися автоматичною документацією; опанувати створення SQLite-бази та виконання SQL-операцій із параметризованими запитами; навчитися перевіряти вхідні дані, формувати моделі відповідей, обробляти помилки й виконувати пов’язані зміни в межах транзакції.

План виконання роботи

1. **Підготовка проєкту:** використання репозиторію з попередніх робіт, додавання `labs/lab03`, встановлення залежностей.
2. **Ознайомлення з теорією:** реляційна модель, SQLite, параметризовані запити, транзакції, HTTP та FastAPI.
3. **Перевірка стилю:** використання наявних налаштувань Ruff.
4. **Виконання індивідуальних завдань:** створення шару роботи з базою даних, розроблення API, перевірка коректності та обробки помилок.
5. **Підготовка звіту:** структура проєкту, схема бази, код, приклади запитів і відповідей, результати Ruff та висновки.

---

1. Робота з Git та структура репозиторію

Крок 1.1: Продовження роботи в репозиторії

1. Використовуйте **той самий репозиторій** `cybersecurity-python-labs`, у якому виконували лабораторні роботи №1 і №2.
2. Додайте директорію `labs/lab03`. Матеріали попередніх робіт збережіть у їхніх наявних директоріях.
3. Використовуйте наявний модуль `shared/student.py` з ПІБ, групою та номером варіанта. Імпортуйте його як пакет без ручного редагування `sys.path`.
4. Код усіх модулів ЛР №3, навчальні початкові дані та інструкцію запуску додайте до Git і завантажте на GitHub.

Крок 1.2: Віртуальне середовище та залежності

Активуйте віртуальне середовище з першої роботи. Якщо його немає на цьому комп’ютері, створіть за інструкцією ЛР №1. Наведені нижче команди виконуйте **з кореня репозиторію**, у середовищі з Python 3.10 або новішим.

```
pip install fastapi uvicorn pydantic ruff
```

Додайте залежності до наявного `requirements.txt`, зберігши пакети попередніх робіт. Для відтворення роботи зафіксуйте перевірені версії використаних пакетів. Приклади моделей у цій роботі розраховані на **Pydantic 2**.

`sqlite3`, `json`, `pathlib`, `logging` та `contextlib` входять до стандартної бібліотеки Python; встановлювати їх через `pip` не потрібно. Роботу з базою реалізуйте без ORM, безпосередньо через `sqlite3`.

Крок 1.3: Організація структури репозиторію

```
cybersecurity-python-labs/
├── .gitignore
├── requirements.txt
├── pyproject.toml           # Якщо вже використовується для Ruff
├── shared/
│   ├── __init__.py
│   └── student.py
└── labs/
    ├── __init__.py
    ├── lab01/              # ЛР №1
    ├── lab02/              # ЛР №2
    └── lab03/
        ├── __init__.py
        ├── main.py         # FastAPI-застосунок і підключення маршрутів
        ├── database.py     # Виконання готового SQL та з'єднання
        ├── schema.sql      # Готова схема для групи варіантів
        ├── repository.py   # Параметризовані SQL-запити
        ├── schemas.py      # Pydantic-моделі запитів і відповідей
        ├── config.py       # Одиниця вимірювання, поріг, обмеження варіанта
        ├── seed.py         # Початкове заповнення бази з JSON
        ├── routers/
        │   ├── __init__.py
        │   ├── categories.py
        │   └── records.py
        ├── data/
        │   ├── seed.json   # Навчальні дані свого варіанта; додати до Git
        │   └── lab3.sqlite3 # Створюється програмою; не додавати до Git
        └── README.md       # Предметна область, запуск і приклади запитів
```

Якщо `labs/__init__.py` відсутній, додайте порожній файл. Імена директорій попередніх робіт у схемі наведено для прикладу; перейменовувати їх не потрібно. Шлях до бази визначайте відносно розташування `database.py`, наприклад через `Path(__file__).resolve().parent`, щоб він не залежав від поточної робочої директорії.

Доповніть `.gitignore`, не видаляючи наявних правил:

```
labs/lab03/data/*.sqlite3
labs/lab03/data/*.sqlite3-*
labs/lab03/data/*.db
labs/lab03/data/*.db-*
```

У репозиторії мають бути **код створення бази та початковий JSON**, завдяки яким базу можна відтворити. Використовуйте лише навчальні дані.

---

2. Теоретичні відомості

2.1. Реляційна модель і SQLite

У реляційній базі дані подаються як таблиці. **Поле** відповідає стовпцю й описує окрему властивість, а **запис** відповідає рядку та містить значення полів конкретного об’єкта.

Первинний ключ (primary key) однозначно визначає запис. **Зовнішній ключ (foreign key)** посилається на запис іншої таблиці. Наприклад, категорія `web` може містити багато записів про знайдені вразливості: це зв’язок «один до багатьох».

SQLite зберігає базу у файлі й не потребує окремого сервера. Модуль `sqlite3` дозволяє відкривати з’єднання, виконувати SQL-запити, отримувати результати та керувати транзакціями.

SQL-операція	Призначення	Приклад для цієї роботи
`CREATE TABLE`	Створення таблиці	Таблиці `categories` і `records`
`INSERT`	Додавання запису	Новий результат перевірки
`SELECT`	Вибірка даних	Список записів певної категорії
`UPDATE`	Оновлення запису	Зміна статусу опрацювання
`DELETE`	Видалення запису	Видалення навчального запису
`JOIN`	Поєднання пов’язаних таблиць	Запис разом із назвою категорії

Обмеження `NOT NULL`, `UNIQUE` та `CHECK` допомагають зберігати узгоджені дані незалежно від того, звідки виконується запит. У SQLite перевірку зовнішніх ключів увімкніть командою `PRAGMA foreign_keys = ON` **для кожного нового з’єднання, до початку транзакції**.

2.2. Параметризовані запити

Параметризований запит (parameterized query) відокремлює SQL-команду від переданих значень:

```
row = connection.execute(
    "SELECT id, code, title FROM records WHERE code = ?",
    (record_code,),
).fetchone()
```

Кома у `(record_code,)` створює кортеж з одного елемента. Не вставляйте введені користувачем значення в SQL через f-рядки, конкатенацію або `%`: це створює ризик **SQL-ін'єкції (SQL injection)**.

Параметри `?` підставляють значення, а не назви таблиць, полів або SQL-оператори. Якщо збираєте фільтри динамічно, SQL-фрагменти мають походити лише з наперед визначених рядків у коді, а всі значення — передаватися окремим списком параметрів.

2.3. Транзакції та помилки

Транзакція (transaction) об'єднує пов'язані зміни: або всі вони зберігаються, або всі скасовуються. `commit()` підтверджує зміни, а `rollback()` скасовує непідтверджені зміни. Наприклад, пакет із трьох записів не повинен зберігатися частково, якщо другий запис порушує `UNIQUE`.

Приклад для стандартного режиму керування транзакціями `sqlite3`:

```
import sqlite3

connection = sqlite3.connect(database_path)
try:
    connection.execute("PRAGMA foreign_keys = ON")
    with connection:
        connection.executemany(
            "INSERT INTO categories (name) VALUES (?)",
            [("web",), ("api",), ("database",)],
        )
except sqlite3.IntegrityError:
    # Контекстний менеджер уже скасував зміни транзакції.
    raise
finally:
    connection.close()
```

Це фрагмент для пояснення транзакції: `database_path` і таблиця мають бути підготовлені раніше. Контекстний менеджер з'єднання `with connection` підтверджує зміни за успіху або скасовує їх за винятку, але **не закриває з'єднання**. У цьому прикладі використано стандартний транзакційний режим; не вмикайте `isolation_level=None` або `autocommit=True`, покладаючись на таку поведінку.

`sqlite3.IntegrityError` повідомляє про порушення обмежень даних. `sqlite3.OperationalError` може виникати через блокування бази або проблеми з доступом до файлу. Деталі технічних помилок записуйте через `logging`, а клієнту повертайте зрозуміле повідомлення без SQL-команд, файлових шляхів і трасування стека.

Для цієї роботи використовуйте синхронні функції маршрутів `def`. Відкривайте, використовуйте й закривайте з'єднання всередині однієї операції шару доступу до даних. Не зберігайте одне глобальне

з’єднання для всіх HTTP-запитів.

2.4. HTTP, ресурси та Web API

Програмний інтерфейс застосунку (Application Programming Interface, API) визначає, як клієнт звертається до функцій і даних сервісу. Web API використовує HTTP-запити та відповіді. У цій роботі дані передаються у JSON.

REST (Representational State Transfer) — архітектурний стиль взаємодії з ресурсами. Ресурс має адресу, а HTTP-метод визначає дію. **CRUD (Create, Read, Update, Delete)** — створення, читання, оновлення та видалення даних.

HTTP-метод	Дія в цій роботі	Приклад
`GET`	Отримати дані	`/records` або `/records/1`
`POST`	Створити запис	`/records`
`PUT`	Повністю замінити змінювані поля наявного запису	`/records/1`
`DELETE`	Видалити запис	`/records/1`

Параметр шляху (path parameter) визначає конкретний ресурс: `1` у `/records/1`. **Параметр запиту (query parameter)** уточнює вибірку: `category_id=1` у `/records?category_id=1`.

Код HTTP	Значення в цій роботі
`200 OK`	Успішне отримання або оновлення
`201 Created`	Успішне створення
`204 No Content`	Успішне видалення; тіло відповіді порожнє
`404 Not Found`	Запис або потрібна категорія не існує
`409 Conflict`	Дублікат унікального значення або категорія містить записи
`422 Unprocessable Entity`	Дані не відповідають моделі чи обмеженням параметрів
`500 Internal Server Error`	Неочікувана помилка роботи сервера

2.5. FastAPI та Pydantic

FastAPI визначає маршрути через декоратори `@app.get`, `@app.post` та інші. `APIRouter` дозволяє розділити маршрути між модулями. На основі маршрутів і моделей FastAPI формує OpenAPI-схему та інтерактивну документацію Swagger UI за адресою `/docs`.

Перевірка вхідних даних (validation) у Pydantic перевіряє типи та задані обмеження. Для створення запису визначають модель запиту, а для результату — модель відповіді, яка задає структуру даних для клієнта.

```
from typing import Literal

from pydantic import BaseModel, ConfigDict, Field


class RecordCreate(BaseModel):
    model_config = ConfigDict(
        str_strip_whitespace=True,
        extra="forbid",
    )

    code: str = Field(min_length=3, max_length=40)
    title: str = Field(min_length=3, max_length=120)
    category_id: int = Field(gt=0)
    value: float = Field(ge=0, le=10, allow_inf_nan=False)
    status: Literal["new", "in_progress", "done"]


class RecordRead(RecordCreate):
    id: int
    category_name: str
```

Цей фрагмент показує модель для варіантів з оцінкою від 0 до 10. Тип і межі `value` змініть за таблицею варіантів. Для лічильників потрібен `int`, для інших вимірювань — скінченний `float` у відповідному діапазоні. Перевірку існування категорії виконує шар роботи з базою, оскільки сама модель не знає її вмісту.

Модель відповіді вкажіть через `response_model` або підтримувану FastAPI анотацію повернення. Для очікуваних помилок використовуйте `HTTPException`, наприклад `raise HTTPException(status_code=404, detail="Запис не знайдено")`.

2.6. Стиль і розподіл відповідальності

Дотримуйтеся PEP 8: відступи з чотирьох пробілів, змістовні назви, впорядковані імпорти, короткі функції. Додавайте анотації типів і короткі docstring до основних функцій.

Маршрути мають обробляти HTTP-запит і формувати відповідь; SQL-запити розміщуйте в `repository.py`, а моделі — у `schemas.py`. Не дублюйте SQL у різних маршрутах. Винятки SQLite перетворюйте на HTTP-помилки в шарі API; модуль роботи з базою не повинен залежати від FastAPI.

---

3. Перевірка коду за допомогою Ruff

Використовуйте налаштування Ruff із ЛР №1, якщо вони вже є в `pyproject.toml` або `ruff.toml`. Не вимикайте наявні правила, щоб приховати помилки нової роботи.

Спочатку перевірте код ЛР №3:

```
ruff check labs/lab03
ruff format --check labs/lab03
```

За потреби застосуйте виправлення й форматування, перегляньте зміни, після чого повторіть перевірку:

```
ruff check --fix labs/lab03
ruff format labs/lab03
ruff check labs/lab03
ruff format --check labs/lab03
```

Перед здачею також запустіть перевірку всього репозиторію:

```
ruff check .
ruff format --check .
```

Виправте знайдені порушення та додайте результати фінальних перевірок у звіт. Ruff перевіряє лише ввімкнені правила: наприклад, перевірка docstring потребує відповідних правил групи `D`. Успішний запуск Ruff не підтверджує правильність SQL, API або транзакцій — їх перевіряють окремо в завданні 3.

---

4. Індивідуальні завдання

Виконайте **всі три завдання** для свого номера варіанта. Для всіх варіантів використовуйте назви таблиць `categories`, `records` і наведені нижче маршрути; предметну область, одиницю вимірювання та обмеження описуйте в документації застосунку.

Завдання 1: SQLite-база та шар доступу до даних

Крок 4.1.1: Використання готового SQL-скрипту

Схема бази даних надається готовою. Самостійно проєктувати таблиці або писати SQL-команди їх створення не потрібно. Для початкового набору підготуйте лише дані свого варіанта за розділом 5.

Створіть файл `labs/lab03/schema.sql`. Скопіюйте до нього послідовно:

1. Спільний початок скрипту зі створенням таблиці `categories`.
2. **Один** готовий блок створення `records`, який відповідає вашому варіанту.
3. Спільне завершення скрипту.

Наведені SQL-блоки використовуйте без змін. Таблиця `categories` містить категорії, а `records` — записи з полями `id`, `code`, `title`, `category_id`, `value`, `status`. Зв’язок між таблицями — «один

до багатьох».

Спільний початок скрипту для всіх варіантів:

```
PRAGMA foreign_keys = ON;

BEGIN TRANSACTION;

CREATE TABLE IF NOT EXISTS categories (
    id INTEGER PRIMARY KEY CHECK (id > 0),
    name TEXT NOT NULL UNIQUE
        CHECK (length(trim(name)) BETWEEN 2 AND 40)
);
```

Варіанти 1–3 та 13–15: оцінки від 0 до 10:

```
CREATE TABLE IF NOT EXISTS records (
    id INTEGER PRIMARY KEY CHECK (id > 0),
    code TEXT NOT NULL UNIQUE
        CHECK (length(trim(code)) BETWEEN 3 AND 40),
    title TEXT NOT NULL
        CHECK (length(trim(title)) BETWEEN 3 AND 120),
    category_id INTEGER NOT NULL
        CHECK (typeof(category_id) = 'integer' AND category_id > 0),
    value REAL NOT NULL
        CHECK (typeof(value) IN ('integer', 'real')
               AND value BETWEEN 0 AND 10),
    status TEXT NOT NULL
        CHECK (status IN ('new', 'in_progress', 'done')),
    FOREIGN KEY (category_id) REFERENCES categories(id)
        ON DELETE RESTRICT
);
```

Варіанти 4–6: кількість подій:

```
CREATE TABLE IF NOT EXISTS records (
    id INTEGER PRIMARY KEY CHECK (id > 0),
    code TEXT NOT NULL UNIQUE
        CHECK (length(trim(code)) BETWEEN 3 AND 40),
    title TEXT NOT NULL
        CHECK (length(trim(title)) BETWEEN 3 AND 120),
    category_id INTEGER NOT NULL
        CHECK (typeof(category_id) = 'integer' AND category_id > 0),
    value INTEGER NOT NULL
        CHECK (typeof(value) = 'integer' AND value >= 0),
    status TEXT NOT NULL
        CHECK (status IN ('new', 'in_progress', 'done')),
    FOREIGN KEY (category_id) REFERENCES categories(id)
        ON DELETE RESTRICT
);
```

Варіанти 7–9: тривалість у хвилинах:

```
CREATE TABLE IF NOT EXISTS records (
    id INTEGER PRIMARY KEY CHECK (id > 0),
    code TEXT NOT NULL UNIQUE
        CHECK (length(trim(code)) BETWEEN 3 AND 40),
    title TEXT NOT NULL
        CHECK (length(trim(title)) BETWEEN 3 AND 120),
    category_id INTEGER NOT NULL
        CHECK (typeof(category_id) = 'integer' AND category_id > 0),
    value REAL NOT NULL
        CHECK (typeof(value) IN ('integer', 'real')
               AND value >= 0
               AND value <= 1.7976931348623157e308),
    status TEXT NOT NULL
        CHECK (status IN ('new', 'in_progress', 'done')),
    FOREIGN KEY (category_id) REFERENCES categories(id)
        ON DELETE RESTRICT
);
```

Варіанти 10–12: відсотки від 0 до 100:

```
CREATE TABLE IF NOT EXISTS records (
    id INTEGER PRIMARY KEY CHECK (id > 0),
    code TEXT NOT NULL UNIQUE
        CHECK (length(trim(code)) BETWEEN 3 AND 40),
    title TEXT NOT NULL
        CHECK (length(trim(title)) BETWEEN 3 AND 120),
    category_id INTEGER NOT NULL
        CHECK (typeof(category_id) = 'integer' AND category_id > 0),
    value REAL NOT NULL
        CHECK (typeof(value) IN ('integer', 'real')
               AND value BETWEEN 0 AND 100),
    status TEXT NOT NULL
        CHECK (status IN ('new', 'in_progress', 'done')),
    FOREIGN KEY (category_id) REFERENCES categories(id)
        ON DELETE RESTRICT
);
```

Спільне завершення скрипту:

```
COMMIT;
```

Скрипт створює дві таблиці з первинними та зовнішнім ключами, перевірками довжин рядків, унікальності, статусу й числового значення. `ON DELETE RESTRICT` забороняє видаляти категорію, яка містить записи. Верхня межа для тривалості у варіантах 7–9 відхиляє нескінченність і не задає прикладного порога реагування.

У SQLite немає окремої команди `CREATE DATABASE`: файл бази створюється під час відкриття з'єднання, якщо ще не існує. Для створення файлу `data/lab3.sqlite3` та виконання готового SQL

використайте наведений код у `labs/lab03/database.py`:

```
import sqlite3
from contextlib import closing
from pathlib import Path


BASE_DIR = Path(__file__).resolve().parent
DATABASE_PATH = BASE_DIR / "data" / "lab3.sqlite3"
SCHEMA_PATH = BASE_DIR / "schema.sql"


def get_connection() -> sqlite3.Connection:
    """Відкрити з’єднання з перевіркою зовнішніх ключів."""
    DATABASE_PATH.parent.mkdir(parents=True, exist_ok=True)
    connection = sqlite3.connect(DATABASE_PATH)
    try:
        connection.execute("PRAGMA foreign_keys = ON")
        return connection
    except sqlite3.Error:
        connection.close()
        raise


def initialize_database() -> None:
    """Створити таблиці за готовим SQL-скриптом."""
    script = SCHEMA_PATH.read_text(encoding="utf-8")
    with closing(get_connection()) as connection:
        try:
            connection.executescript(script)
        except sqlite3.Error:
            connection.rollback()
            raise


if __name__ == "__main__":
    initialize_database()
```

Запустіть із кореня репозиторію:

```
python -m labs.lab03.database
```

Повторний запуск зберігає наявні таблиці та дані завдяки `IF NOT EXISTS`. Ця команда не заповнює базу: навчальні дані готуються в наступному кроці. До Git додайте `schema.sql` і `database.py`; робочий файл SQLite не додавайте.

Готові функції `get_connection()` та `initialize_database()` можна використовувати в інших модулях. Ініціалізацію виконуйте до початкового заповнення або під час запуску API, **поза транзакцією додавання записів**. Перевірка зовнішніх ключів має бути ввімкнена для кожного з’єднання, тому одного `PRAGMA` у файлі схеми недостатньо для всіх наступних запитів.

Крок 4.1.2: Початкові дані

1. Після створення бази готовим скриптом підготуйте `data/seed.json` за правилами розділу 5 для свого варіанта. Самостійно визначте лише навчальні дані; схему таблиць залиште наданою.
2. У `seed.py` реалізуйте читання JSON, перевірку його даних і початкове заповнення бази.
3. Для нової порожньої бази додайте **три категорії та вісім записів** в одній транзакції. Вставляйте категорії з `id`, визначеними в JSON; у звичайному API ідентифікатори створює база.
4. Якщо початкові дані некоректні або будь-яке додавання порушує обмеження, скасуйте весь пакет змін і виведіть зрозуміле повідомлення.
5. Повторне заповнення непорожньої бази має завершуватися повідомленням без зміни даних і без дублікатів. Не використовуйте `INSERT OR IGNORE`, щоб приховувати помилки в початковому наборі.

Запуск із кореня репозиторію:

```
python -m labs.lab03.seed
```

Крок 4.1.3: SQL-операції

У `repository.py` реалізуйте функції для:

1. Створення категорії, отримання списку категорій і видалення порожньої категорії.
2. Додавання запису й отримання запису за `id`.
3. Отримання списку записів із фільтрами та поділом на сторінки.
4. Повного оновлення змінюваних полів запису; `id` має залишатися незмінним.
5. Видалення запису.
6. Пакетного додавання записів у межах однієї транзакції.

Усі введені значення передавайте через параметри SQL. У результатах читання записів використовуйте `JOIN`, щоб повернути також `category_name`.

Після кожної операції закривайте з’єднання. В операціях зміни даних забезпечте підтвердження або скасування транзакції. Для перевірки існування запису не покладайтеся лише на успішне виконання `UPDATE` чи `DELETE`: обробіть ситуацію, коли відповідного `id` немає.

Завдання 2: Web API за допомогою FastAPI

Крок 4.2.1: Застосунок і документація

1. У `main.py` створіть FastAPI-застосунок із назвою, описом предметної області та одиниці вимірювання.
2. Підключіть маршрути через `APIRouter` із модулів `routers`.

3. Під час запуску створюйте відсутні таблиці, але не перезаписуйте дані й не заповнюйте базу автоматично за кожного запиту. Ініціалізацію можна виконати через контекст життєвого циклу `lifespan`.
4. У Swagger UI мають бути зрозумілі назви операцій, описи параметрів, моделі даних і можливі відповіді `404` та `409` для відповідних маршрутів.

Крок 4.2.2: Моделі та вхідні дані

Визначте окремі моделі `CategoryCreate`, `CategoryRead`, `RecordCreate`, `RecordReplace` та `RecordRead`. `RecordReplace` може успадковувати `RecordCreate`: усі змінювані поля в `PUT` обов'язкові.

Перевіряйте:

- довжини рядків і відсутність порожнього тексту після обрізання пробілів;
- додатні ідентифікатори;
- тип і діапазон `value` за варіантом;
- статус із визначеного набору;
- відсутність невідомих полів у тілі запиту через `extra="forbid"`;
- кількість елементів пакета: від 1 до 100.

Для моделей читання запису повертайте `id` і `category_name` разом з усіма змінюваними полями. Клієнт не повинен задавати `id` під час створення або заміни запису.

Крок 4.2.3: Обов'язкові маршрути

Метод і маршрут	Призначення	Успішна відповідь
`GET /ping`	Перевірка доступності API	`200`, `{"status": "ok"}`
`GET /student`	Дані з `shared/student.py`	`200`, ПІБ, група та варіант
`GET /categories`	Список категорій за зростанням `id`	`200`, список `CategoryRead`
`POST /categories`	Створення категорії	`201`, `CategoryRead`
`DELETE /categories/{category_id}`	Видалення порожньої категорії	`204`, без тіла
`GET /records`	Список із фільтрами	`200`, список `RecordRead`
`GET /records/{record_id}`	Один запис	`200`, `RecordRead`

Метод і маршрут	Призначення	Успішна відповідь
`POST /records`	Створення запису	`201`, `RecordRead`
`PUT /records/{record_id}`	Повна заміна змінюваних полів	`200`, `RecordRead`
`DELETE /records/{record_id}`	Видалення запису	`204`, без тіла
`POST /records/bulk`	Пакетне створення з атомарним збереженням	`201`, список створених `RecordRead`

`GET /student` має повертати ключі `student_name`, `group_name`, `variant_number`. `POST /records/bulk` приймає **JSON-масив** об'єктів `RecordCreate`.

Параметри `record_id` і `category_id` у шляхах мають бути додатними цілими числами. Неіснуючий запис або категорія — `404`. Дублікат `code` чи назви категорії, а також спроба видалити непорожню категорію — `409`. Некоректна модель або параметр — `422`.

Якщо створення чи оновлення запису посилається на неіснуючу категорію, поверніть `404`. Якщо `GET /records` фільтрує за додатним `category_id`, якого немає, поверніть порожній список. Так само поверніть порожній список для інших фільтрів без збігів.

Крок 4.2.4: Параметри списку

Реалізуйте такі параметри `GET /records`:

Параметр	Тип і обмеження	Дія
`category_id`	Необов'язковий `int > 0`	Лише записи категорії
`status`	Необов'язковий дозволений статус	Лише записи з цим статусом
`min_value`	Необов'язковий, у діапазоні `value` свого варіанта	`value >= min_value`
`q`	Необов'язковий рядок довжиною 1–120	Буквальний підрядок у `title`, з урахуванням регістру
`limit`	`int`, 1–100, типово 20	Максимальна кількість записів
`offset`	`int >= 0`, типово 0	Кількість пропущених записів

Якщо задано кілька фільтрів, об'єднуйте їх через `AND`. Результати сортуйте за `id` у зростаючому порядку, щоб поділ на сторінки був передбачуваним. Для буквального пошуку підрядка можна

використати `instr(title, ?) > 0`; символи `%` та `_` у такому пошуку не є шаблонами.

Крок 4.2.5: Пакет і транзакція

1. Перевірте весь JSON-масив через Pydantic перед записом у базу.
2. Перевірте посилання на категорії та додайте записи в **одній транзакції**. Якщо будь-який елемент має дублікат `code`, неіснуючу категорію або порушує обмеження, не зберігайте жодного елемента пакета.
3. Дублікат у базі або всередині пакета має повертати `409`; неіснуюча категорія — `404`; помилка моделі — `422`.
4. Не використовуйте окремий `commit()` після кожного елемента.

Крок 4.2.6: Запуск

```
python -m labs.lab03.seed
uvicorn labs.lab03.main:app --reload --host 127.0.0.1 --port 8000
```

Відкрийте Swagger UI і OpenAPI-схему. Режим `--reload` призначений для розроблення.

Завдання 3: Перевірка API, збереження даних і транзакцій

Виконайте перевірки через Swagger UI або `curl`. Автоматизовані перевірки можна додати за бажанням; для обов'язкової частини достатньо відтворюваних запитів і результатів у звіті.

№	Сценарій	Що продемонструвати
1	Початкове заповнення та повторний запуск `seed.py`	3 категорії, 8 записів; повторний запуск не додає дублікатів
2	`GET /ping`, `/student`, `/categories`	Правильні дані та `200`
3	Створення, читання, `PUT` і видалення окремого запису	Повний цикл CRUD і відповідні HTTP-коди
4	Фільтри, два фільтри одночасно, `limit` і `offset`	Вибірка відповідає умовам і має стабільний порядок
5	Від'ємне `value`, порожній `title`, недозволений статус, `limit=0`	`422`, база не змінюється
6	Значення за межами верхньої межі; для лічильників — `value=1.5`	`422`; для варіантів без верхньої межі перевірте неправильний тип
7	Неіснуючий `record_id`; запис із неіснуючим `category_id`	`404`

№	Сценарій	Що продемонструвати
8	Повторний `code` і повторна назва категорії	`409`, попередні дані зберігаються
9	Видалення непорожньої та порожньої категорій	Відповідно `409` і `204`; зв’язки не порушуються
10	Коректний пакет із двох нових записів	`201`, обидва записи збережено
11	Пакет: перший запис новий, другий має `code` із бази	`409`; перший запис також не збережено
12	Пакет: перший запис новий, другий має неіснуючу категорію	`404`; жоден новий запис не збережено
13	Пошук `q=' OR 1=1 --`	Рядок обробляється як дані; API працює, записи не змінюються
14	Створення запису та перезапуск сервера	Запис доступний після перезапуску

Для сценаріїв 11 і 12 зафіксуйте кількість записів до та після запиту і перевірте відсутність `code` першого елемента. Якщо перевіряєте через `GET /records`, врахуйте `limit` і за потреби отримайте всі сторінки за допомогою `offset`.

Приклади виконання запитів

Наведені команди розраховані на Bash. У Windows можна виконати ті самі запити через Swagger UI.

```
curl -i "http://127.0.0.1:8000/ping"
curl -i "http://127.0.0.1:8000/records?category_id=1&status=new&limit=5&offset=0"
```

Приклад створення запису для варіанта 1:

```
curl -i -X POST "http://127.0.0.1:8000/records" \
  -H "Content-Type: application/json" \
  -d '{"code":"V01-TEST","title":"Навчальна
знахідка","category_id":1,"value":7.5,"status":"new"}'
```

Очікувано: `201 Created`; відповідь містить передані поля, створений `id` і `category_name: "web"`. Використайте отриманий `id` для читання, заміни та видалення цього запису. У `PUT` передайте всі п’ять змінюваних полів.

Для перевірки скасування пакета у варіанті 1, коли `V01-01` уже є в базі:

```
curl -i -X POST "http://127.0.0.1:8000/records/bulk" \
  -H "Content-Type: application/json" \
  -d '[{"code":"V01-ROLLBACK","title":"Перший
елемент","category_id":1,"value":4.0,"status":"new"},{"code":"V01-01","title":"Дублікат
коду","category_id":2,"value":5.0,"status":"new"}]'
```

Очікувано: `409 Conflict`; запис `V01-ROLLBACK` відсутній, кількість записів не змінилася. Наведений приклад перевіряє атомарність результату; реалізація також має містити одну транзакцію для вставлення всього пакета.

---

5. Вхідні дані та 15 варіантів завдань

5.1. Спільні правила підготовки даних

Усі потрібні початкові дані наведено нижче. Додаткові журнали подій, зовнішні API або архіви не потрібні. Значення є навчальними й не описують реальну інфраструктуру.

Кожен варіант задає предметну область, три назви категорій, одиницю вимірювання, поріг і вісім значень. Записи побудуйте за таким правилом:

Номер запису `i`	**`category_id`**	**`status`**
1	1	`new`
2	2	`in_progress`
3	3	`done`
4	1	`new`
5	2	`done`
6	3	`in_progress`
7	1	`done`
8	2	`new`

- Категорії отримують `id` 1, 2, 3 у порядку їх наведення.
- `code` має формат `V{варіант:02d}-{i:02d}`, наприклад `V03-07`.
- `title` складається з назви запису свого варіанта і номера, наприклад `Перевірка сервісу 01`.
- `value` береться з відповідної позиції списку значень.
- `id` запису в JSON не задавайте: його створює SQLite.

Схема `seed.json`: об’єкт із двома масивами, `categories` і `records`. Наприклад, перша категорія та перший запис варіанта 1 мають вигляд:

```
{
  "categories": [
    {"id": 1, "name": "web"}
  ],
  "records": [
    {
      "code": "V01-01",
      "title": "Знахідка вебаудиту 01",
      "category_id": 1,
      "value": 9.1,
      "status": "new"
    }
  ]
}
```

Це приклад структури, а не повний набір. До свого файлу додайте всі три категорії та вісім записів. JSON можна підготувати вручну або сформувати коротким допоміжним кодом. Статуси описують опрацювання запису й не змінюють його числове значення.

5.2. Типи значень і обмеження

Варіанти	Зміст `value`	Тип у моделі	Межі	Тип у SQLite	`unit`
1–3	Навчальна оцінка небезпеки вразливості	`float`	0–10 включно	`REAL`	`score`
4–6	Кількість подій у спостереженні	`int`	Не менше 0	`INTEGER`	`events`
7–9	Час реагування або відновлення	`float`	Не менше 0	`REAL`	`minutes`
10–12	Виміряна частка використання ресурсу	`float`	0–100 включно	`REAL`	`percent`
13–15	Навчальна оцінка ризику знахідки аудиту	`float`	0–10 включно	`REAL`	`score`

Для `float` відхиляйте нескінченні значення та `NaN`. У варіантах 4–6 дробове число на кшталт `1.5` має бути помилкою. Оцінки ризику та небезпеки тут задані без розрахунку за окремою стандартизованою методикою.

Варіант 1: Реєстр вразливостей вебзастосунку

Розробіть сервіс обліку знахідок перевірки вебзастосунку. Категорія визначає його компонент, `value` — навчальну оцінку небезпеки, `status` — стан опрацювання знахідки.

- **Категорії:** `web`, `api`, `database`.
- **Назва запису для `title`:** `Знахідка вебаудиту`.
- **Поріг:** `7.0`.
- **Значення:** `[9.1, 5.4, 3.2, 7.5, 8.0, 2.4, 6.3, 7.1]`.

Варіант 2: Реєстр вразливостей мережевих сервісів

Розробіть сервіс обліку результатів навчальної перевірки мережевих сервісів. Категорія визначає групу сервісів, `value` — оцінку небезпеки, `status` — стан опрацювання.

- **Категорії:** `mail`, `vpn`, `dns`.
- **Назва запису:** `Перевірка сервісу`.
- **Поріг:** `6.0`.
- **Значення:** `[6.8, 4.1, 8.2, 5.0, 7.4, 3.6, 9.0, 6.0]`.

Варіант 3: Реєстр вразливостей робочих станцій

Розробіть сервіс обліку знахідок перевірки робочих станцій. Категорія визначає тип програмного забезпечення, `value` — оцінку небезпеки, `status` — стан опрацювання.

- **Категорії:** `os`, `browser`, `office`.
- **Назва запису:** `Знахідка станції`.
- **Поріг:** `8.0`.
- **Значення:** `[8.5, 2.7, 6.1, 9.4, 4.8, 7.0, 8.0, 5.5]`.

Варіант 4: Облік подій автентифікації

Розробіть сервіс збереження підсумків спостереження за автентифікацією. Один запис описує кількість подій певного типу за навчальний інтервал; `status` визначає стан перевірки спостереження.

- **Категорії:** `failed_login`, `blocked_account`, `mfa_failure`.
- **Назва запису:** `Інтервал автентифікації`.
- **Поріг:** `20`.
- **Значення:** `[25, 8, 3, 40, 12, 5, 18, 22]`.

Варіант 5: Облік спрацювань засобів захисту

Розробіть сервіс збереження кількості спрацювань засобів захисту за навчальні інтервали. Категорія визначає джерело спрацювання, `status` — стан перевірки підсумкового запису.

- **Категорії:** `firewall`, `ids`, `antivirus`.
- **Назва запису:** `Підсумок спрацювань`.
- **Поріг:** `15`.
- **Значення:** `[10, 35, 7, 22, 15, 4, 28, 9]`.

Варіант 6: Облік подій поштової безпеки

Розробіть сервіс обліку підсумків перевірки повідомлень. `value` містить кількість виявлених подій у спостереженні, а `status` — стан опрацювання цього спостереження.

- **Категорії:** `phishing`, `malicious_attachment`, `spam`.
- **Назва запису:** `Спостереження пошти`.
- **Поріг:** `10`.
- **Значення:** `[12, 4, 30, 8, 15, 45, 10, 6]`.

Варіант 7: Облік часу реагування на інциденти

Розробіть сервіс збереження вимірювань часу реагування. Категорія визначає тип інциденту, `value` — тривалість у хвилинах, `status` — стан перевірки запису.

- **Категорії:** `account`, `network`, `endpoint`.
- **Назва запису:** `Вимірювання реагування`.
- **Поріг:** `30.0`.
- **Значення:** `[12.0, 45.0, 20.0, 35.0, 60.0, 18.0, 30.0, 25.0]`.

Варіант 8: Облік часу відновлення сервісів

Розробіть сервіс обліку навчальних вимірювань відновлення. Категорія визначає сервіс, `value` — час у хвилинах, `status` — стан перевірки вимірювання.

- **Категорії:** `database`, `web_service`, `file_storage`.
- **Назва запису:** `Вимірювання відновлення`.
- **Поріг:** `60.0`.
- **Значення:** `[75.0, 20.0, 40.0, 90.0, 35.0, 65.0, 55.0, 60.0]`.

Варіант 9: Облік часу опрацювання повідомлень

Розробіть сервіс обліку тривалості перевірки повідомлень про події безпеки. Категорія визначає джерело повідомлення, `value` — тривалість у хвилинах, `status` — стан перевірки вимірювання.

- **Категорії:** `siem`, `helpdesk`, `monitoring`.
- **Назва запису:** `Вимірювання опрацювання`.
- **Поріг:** `15.0`.

- **Значення:** `[5.0, 18.0, 12.0, 25.0, 9.0, 30.0, 15.0, 7.0]`.

Варіант 10: Облік навантаження серверних ресурсів

Розробіть сервіс обліку вимірювань використання серверних ресурсів. Категорія визначає ресурс, `value` — використання у відсотках, `status` — стан перевірки спостереження.

- **Категорії:** `cpu`, `memory`, `disk`.
- **Назва запису:** `Вимірювання сервера`.
- **Поріг:** `80.0`.
- **Значення:** `[82.0, 65.0, 40.0, 91.0, 78.0, 55.0, 80.0, 88.0]`.

Варіант 11: Облік завантаження каналів зв’язку

Розробіть сервіс обліку вимірювань завантаження каналів. Категорія визначає канал, `value` — частку використання його пропускної здатності у відсотках, `status` — стан перевірки вимірювання.

- **Категорії:** `internet`, `vpn`, `backup_link`.
- **Назва запису:** `Вимірювання каналу`.
- **Поріг:** `70.0`.
- **Значення:** `[72.0, 50.0, 25.0, 85.0, 68.0, 30.0, 70.0, 92.0]`.

Варіант 12: Облік використання сховищ

Розробіть сервіс обліку вимірювань заповнення сховищ. Категорія визначає тип сховища, `value` — частку зайнятого простору у відсотках, `status` — стан перевірки вимірювання.

- **Категорії:** `logs`, `backups`, `archive`.
- **Назва запису:** `Вимірювання сховища`.
- **Поріг:** `85.0`.
- **Значення:** `[90.0, 60.0, 45.0, 95.0, 83.0, 70.0, 85.0, 88.0]`.

Варіант 13: Реєстр ризиків хмарної конфігурації

Розробіть сервіс обліку знахідок аудиту хмарної конфігурації. Категорія визначає напрям перевірки, `value` — навчальну оцінку ризику, `status` — стан опрацювання знахідки.

- **Категорії:** `iam`, `storage`, `network`.
- **Назва запису:** `Знахідка хмарного аудиту`.
- **Поріг:** `7.0`.
- **Значення:** `[8.0, 6.5, 4.0, 9.0, 7.5, 3.0, 7.0, 5.0]`.

Варіант 14: Реєстр ризиків контейнерної конфігурації

Розробіть сервіс обліку результатів перевірки конфігурації контейнерного середовища. Категорія визначає напрям перевірки, `value` — навчальну оцінку ризику, `status` — стан опрацювання.

- **Категорії:** `image`, `runtime`, `secrets`.
- **Назва запису:** `Знахідка контейнерного аудиту`.
- **Поріг:** `6.0`.
- **Значення:** `[6.0, 8.0, 9.5, 4.0, 7.0, 5.5, 3.0, 6.5]`.

Варіант 15: Реєстр ризиків налаштування робочих станцій

Розробіть сервіс обліку знахідок аудиту налаштувань робочих станцій. Категорія визначає напрям перевірки, `value` — навчальну оцінку ризику, `status` — стан опрацювання.

- **Категорії:** `updates`, `permissions`, `protection`.
- **Назва запису:** `Знахідка аудиту станції`.
- **Поріг:** `8.0`.
- **Значення:** `[8.5, 5.0, 7.0, 9.0, 6.0, 8.0, 4.5, 9.5]`.

---

6. Підготовка звіту та здача роботи

У звіті наведіть:

1. Короткий опис предметної області й значення полів, одиниці вимірювання та порога.
2. Структуру директорій проєкту та посилання на репозиторій.
3. Надану схему двох таблиць із ключами й обмеженнями та готовий SQL-скрипт, використаний для свого варіанта.
4. Лістинг основних модулів: створення бази, SQL-операції, моделі, маршрути й транзакція пакетного додавання.
5. Початкові дані свого варіанта та результати перевірок із завдання 3: запит, HTTP-код, відповідь або стан бази.
6. Демонстрацію скасування пакета з підтвердженням, що жоден його елемент не збережено.
7. Результати `ruff check` і `ruff format --check`.
8. Скріншоти і приклади виконання скриптів і запитів.
9. Висновки: що реалізовано, як забезпечено перевірку даних, захист SQL-запитів і цілісність бази.

У `labs/lab03/README.md` додайте встановлення залежностей, команду початкового заповнення, команду запуску API та приклади основних запитів. За цими інструкціями має бути можливо відтворити застосунок після клонування репозиторію.

Завантажте код на GitHub, наприклад:

```
git status
git add labs/lab03 requirements.txt .gitignore
git commit -m "Add lab 3 FastAPI and SQLite application"
git push
```

Якщо додавали `labs/__init__.py` або змінювали конфігурацію Ruff, також додайте ці файли до коміту. Перед комітом перевірте список змін: файл бази та віртуальне середовище не мають потрапити в репозиторій.

На захисті продемонструйте: запуск із відтворюваної бази, один цикл CRUD, фільтрацію, помилку перевірки даних, конфлікт унікальності та скасування всього пакета. Поясніть SQL-запити й розподіл коду між модулями.

---

7. Контрольні питання для обговорення (Захист роботи)

1. Що означають таблиця, запис і поле? Які сутності описують ваші дві таблиці та який зв’язок між ними?
2. Чим відрізняються первинний і зовнішній ключі? Що станеться під час спроби створити запис із неіснуючою категорією?
3. Чим файлова база SQLite відрізняється від CSV або JSON? Які можливості база додає для пошуку й підтримання узгодженості даних?
4. Параметризовані запити: як вони захищають від SQL-ін’єкції? Чому `?` не можна використати для довільної назви таблиці або поля?
5. Чому пакетне додавання потребує однієї транзакції? Що станеться, якщо виконувати `commit()` після кожного запису?
6. Що робить `with connection` у вашому коді? Чому після нього з’єднання все одно потрібно закрити?
7. Як співвідносяться `GET`, `POST`, `PUT`, `DELETE` з SQL-операціями? Чому в цій роботі `PUT` потребує всіх змінюваних полів?
8. Чим відрізняються параметри шляху, параметри запиту та тіло запиту? Покажіть їх у власних маршрутах.
9. Pydantic: що перевіряє модель запиту?
10. Для чого потрібна `response_model`?
11. Чому модель створення не повинна приймати `id` запису?
12. HTTP-коди: наведіть приклади `201`, `204`, `404`, `409` і `422` зі своєї роботи. Чому для `204` не повертають JSON-тіло?
13. Чому початковий JSON та код створення бази зберігаються в Git, а робочий файл SQLite — ні?

Корисні посилання (Теоретичні джерела)

Python і SQLite

- Модуль sqlite3 — офіційна документація Python українською.
- Модуль json — читання початкових даних.
- Модуль pathlib — робота зі шляхами.
- Обробка винятків у Python.
- Модуль logging — журналювання технічних помилок.
- SQL у SQLite — офіційний довідник.
- Зовнішні ключі SQLite.
- Транзакції SQLite.

FastAPI та Pydantic

- FastAPI — посібник користувача українською.
- Параметри шляху.
- Параметри запиту.
- Тіло запиту та Pydantic-моделі.
- Моделі відповідей.
- Обробка HTTP-помилок.
- Структура застосунку та APIRouter.
- Ініціалізація через lifespan.
- Pydantic — обмеження полів.

Стиль коду

- PEP 8 — рекомендації щодо стилю Python.
- Ruff — посібник і команди перевірки.
- Ruff — конфігурація та вибір правил.